PySpark Notebook – Bronze to Silver Layer with Transformations & Upserts

Lab Name:

Create PySpark Notebook – Bronze to Silver Layer with Transformations & Upserts

Lab Summary:

In this lab, you will:

- Create a PySpark notebook
- Read raw CSV data from Bronze volume
- Perform transformations using PySpark
- Apply aggregations
- Create visualizations inside notebook
- Add processing timestamp column
- Write data to Silver layer in Delta format
- Implement **Upsert (Merge)** logic using Delta Lake
- Build complete Silver layer tables for:
  - Customers
  - Products
  - Stores
  - Sales

This lab establishes the **Silver (Enriched) Layer** using PySpark and Delta Lake.

Prerequisites:

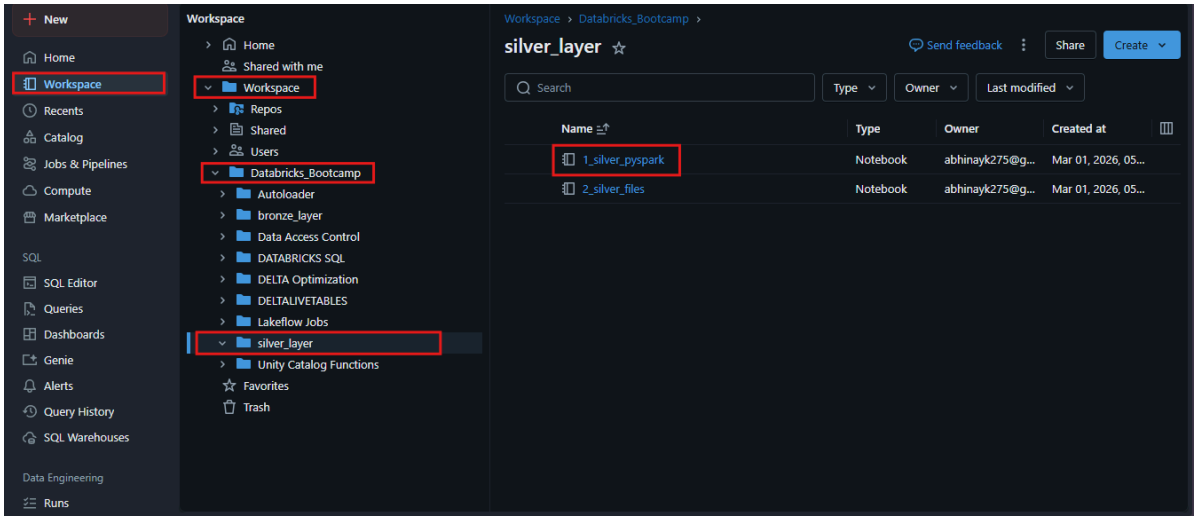
- Unity Catalog enabled workspace
- `databricksansh` catalog created
- Bronze schema and volume created
- Raw CSV files uploaded in Bronze volume
- Serverless compute running
- Silver schema created

Steps to Complete the Lab

Task 1: Create PySpark Notebook

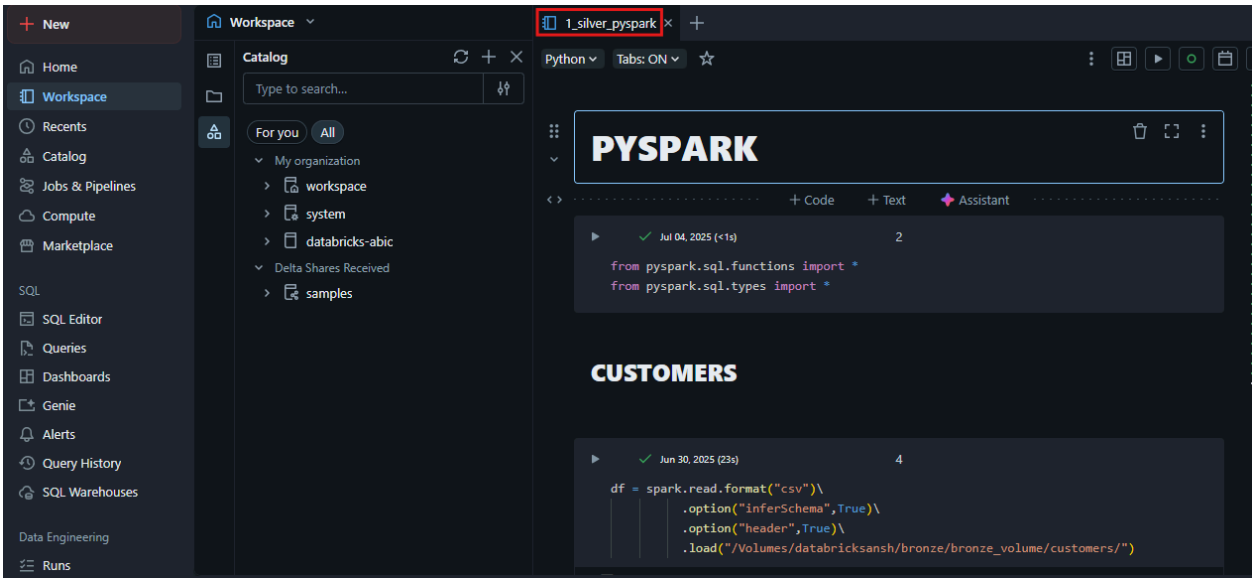
Step 1: Create Notebook

- Click **Workspace**
- Click Databricks_Bootcamp → **Silver_layer** → **1_silver_pyspark**



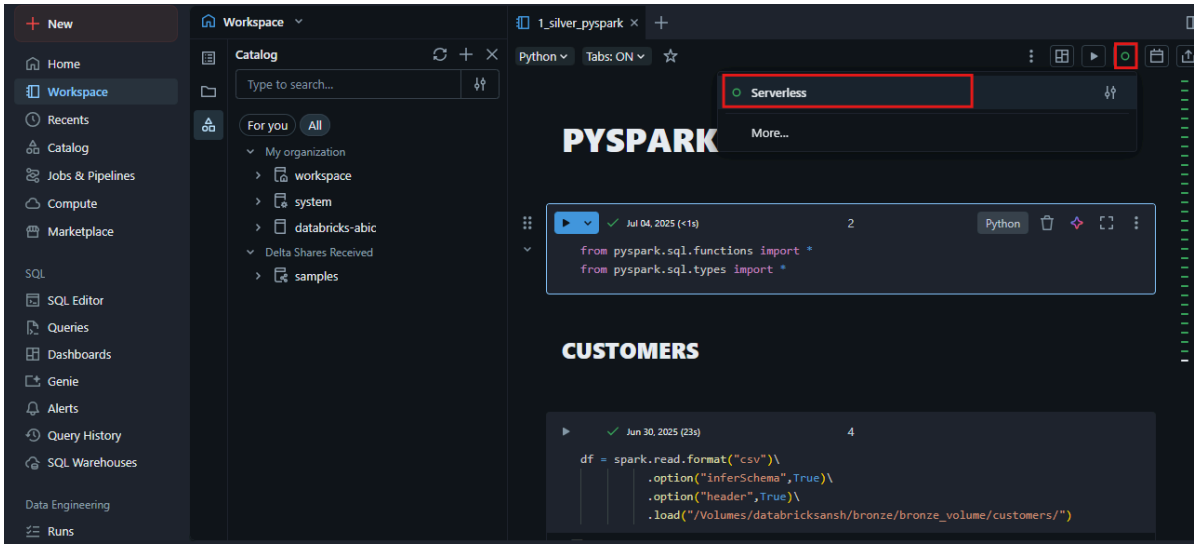
- Open Notebook:

```
1_silver_pyspark
```



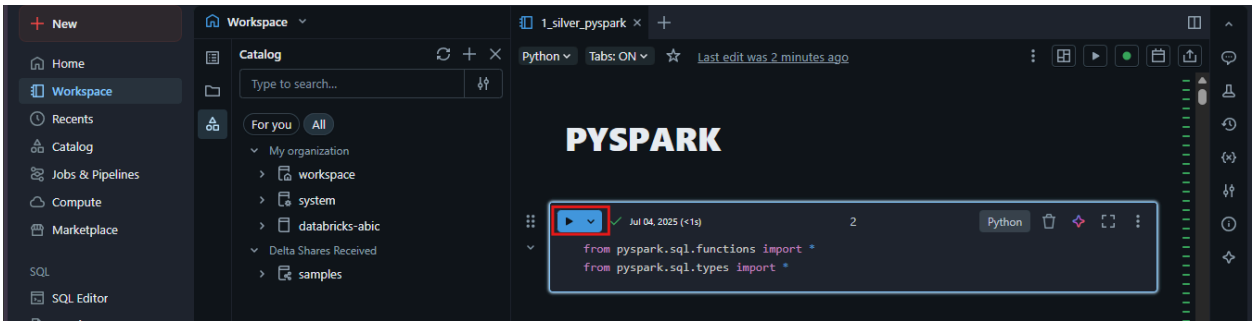
Step 2: Attach Compute

- Connect to **Serverless Compute**



Task 2: Import Required Libraries

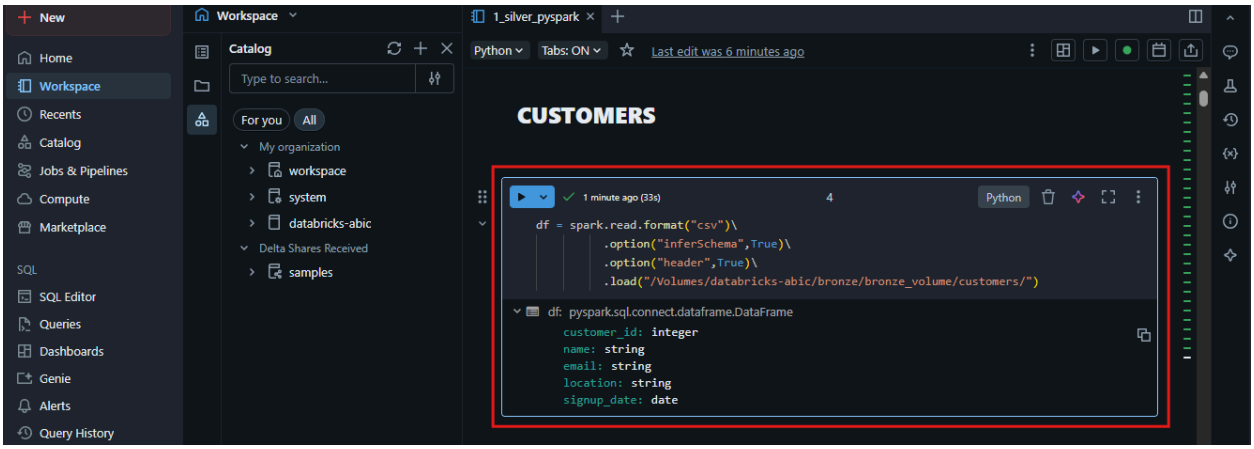
```
from pyspark.sql.functions import *
from pyspark.sql.types import *
from delta.tables import DeltaTable
```



Task 3: Process Customers Data (Bronze → Silver)

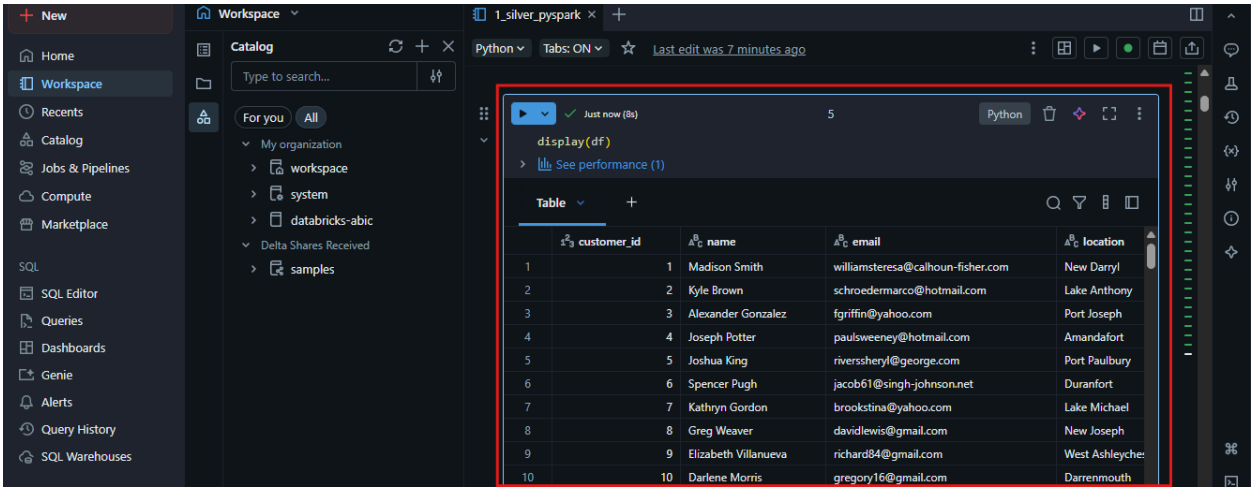
Step 3.1: Read Customers CSV from Volume

```
df = spark.read \
    .format("csv") \
    .option("inferSchema", True) \
    .option("header", True) \
    .load("/Volumes/databricks-abic/bronze/bronze_volume/cust
omers/")
```



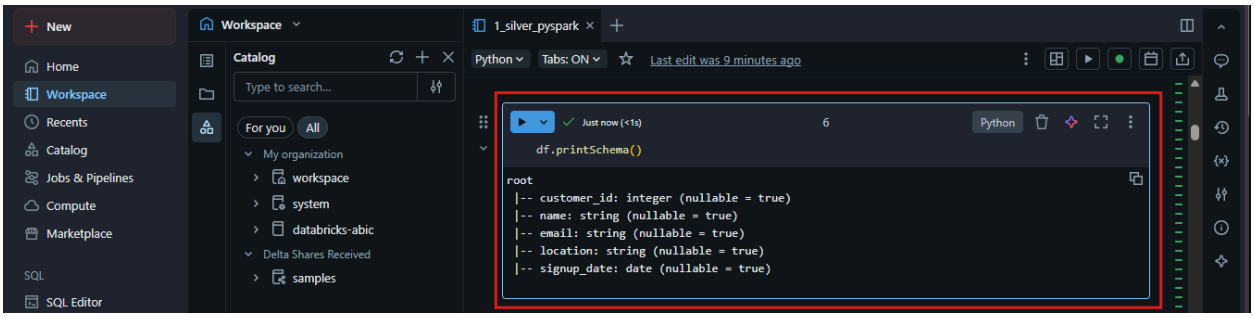
Display Data

```
display(df)
```



Check Schema

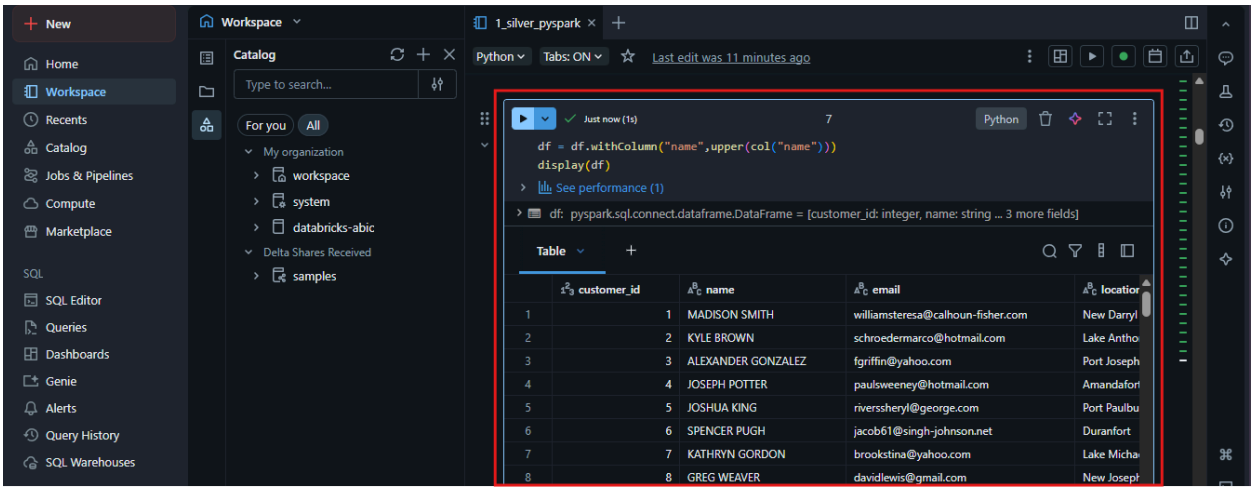
```
df.printSchema()
```



Step 3.2: Transformations

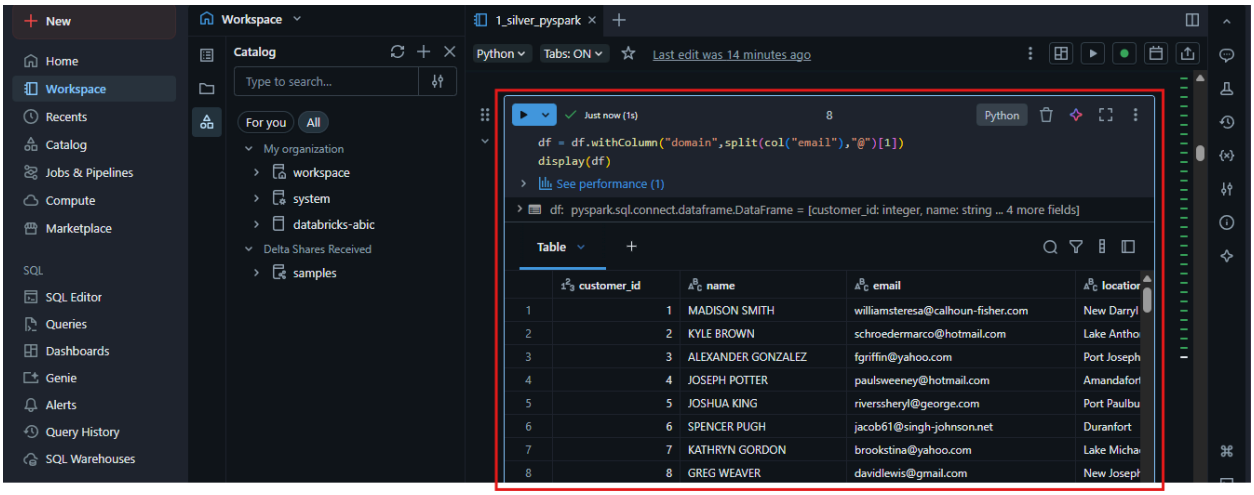
Convert Name to Uppercase

```
df = df.withColumn("name", upper(col("name")))
```



Extract Email Domain

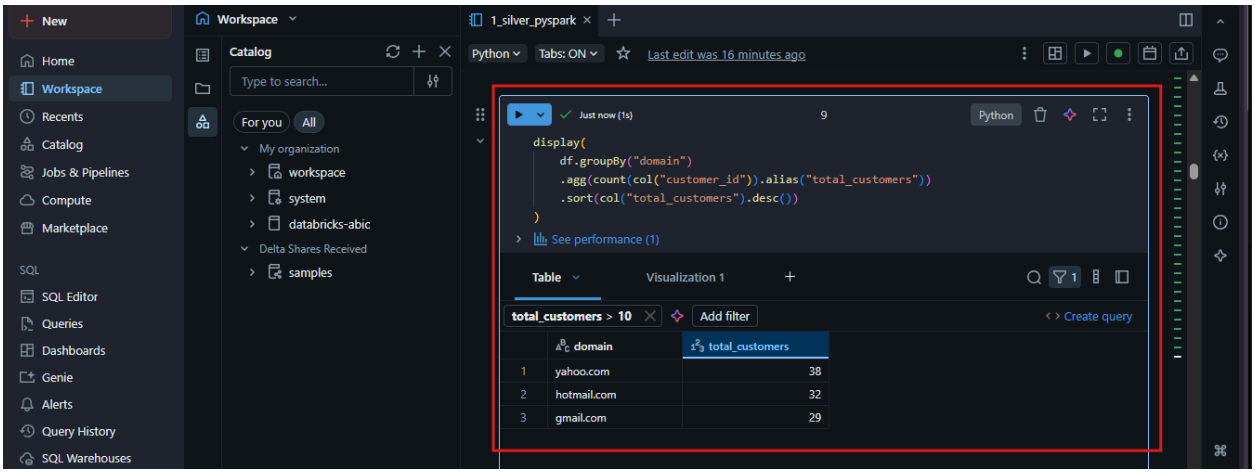
```
df = df.withColumn(
    "domain",
    split(col("email"), "@")[1]
)
```



Step 3.3: Perform Aggregation

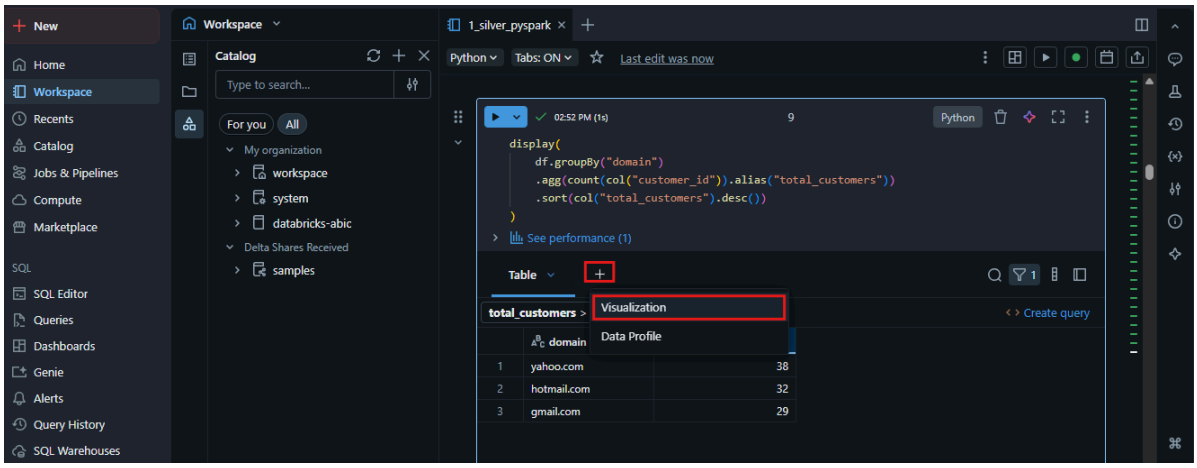
Count Customers by Domain

```
display(
    df.groupBy("domain")
      .agg(count("customer_id").alias("total_customers"))
      .sort(col("total_customers").desc())
)
```

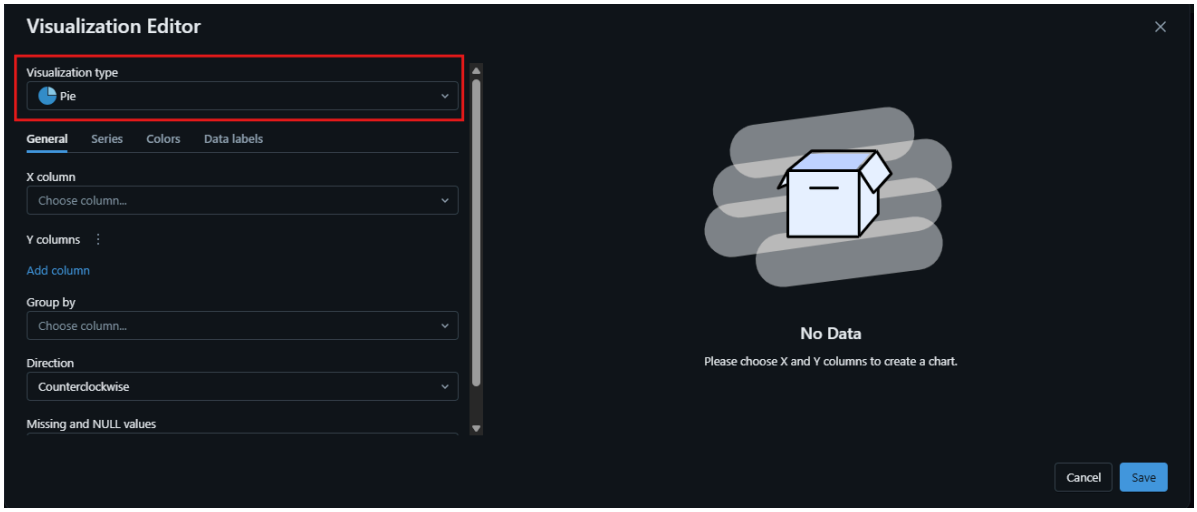


Step 3.4: Create Visualization

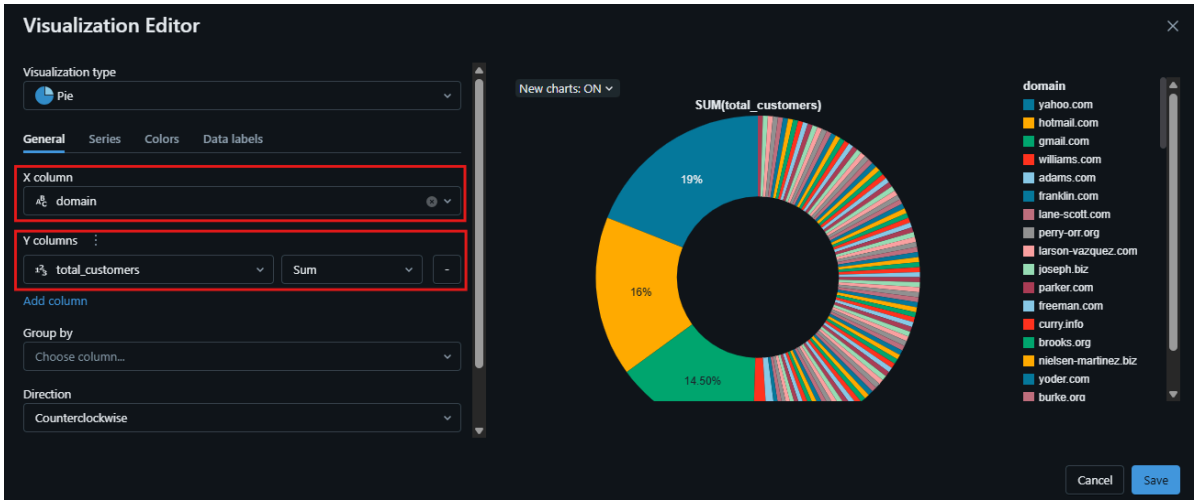
1. Click **+** → **Visualization**



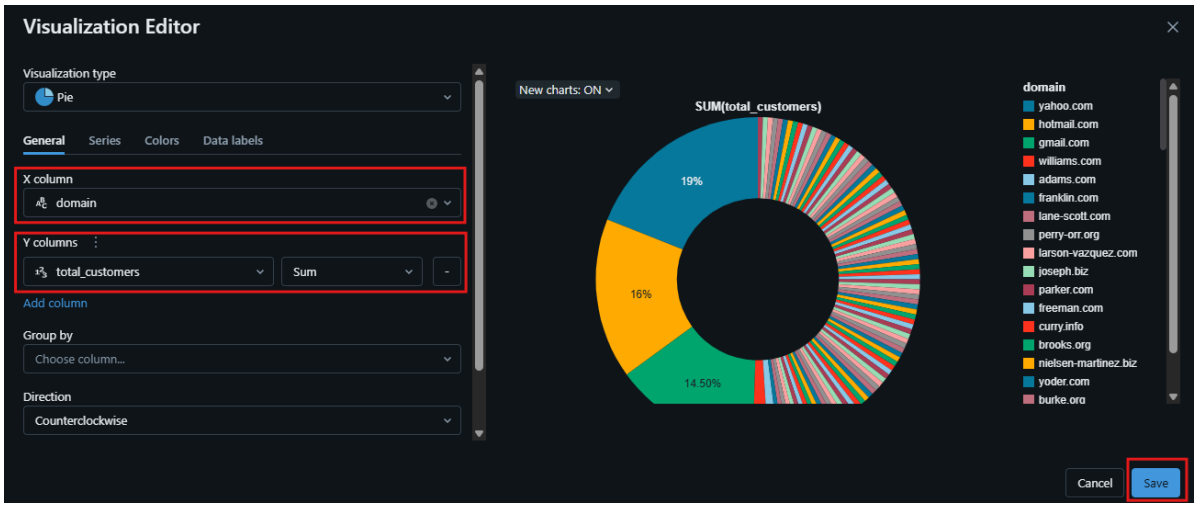
2. Choose **Pie Chart**



3. X-axis → domain —→ Y-axis → total_customers

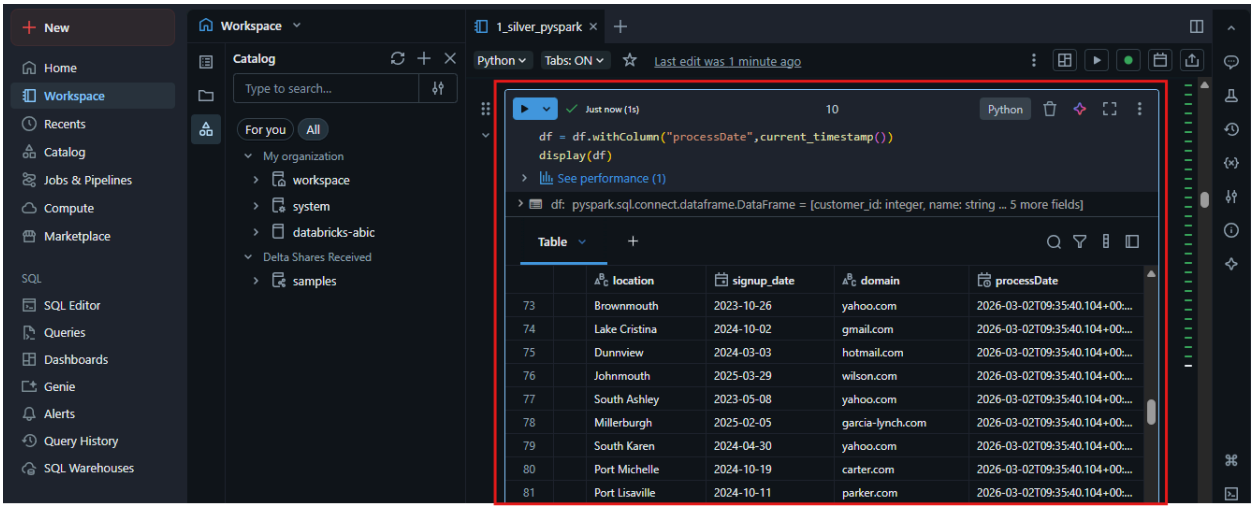


4. Save visualization



Step 3.5: Add Processing Timestamp

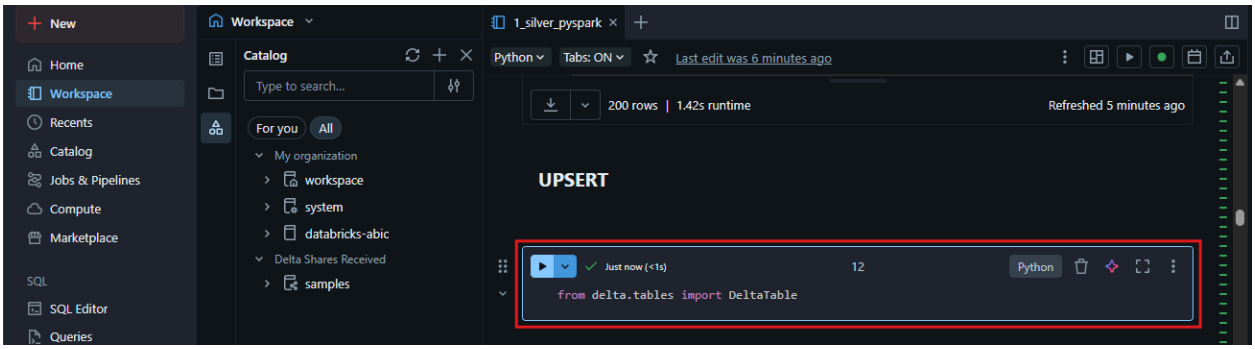
```
df = df.withColumn("process_date", current_timestamp())
```



Step 3.6: Write to Silver with Upsert Logic

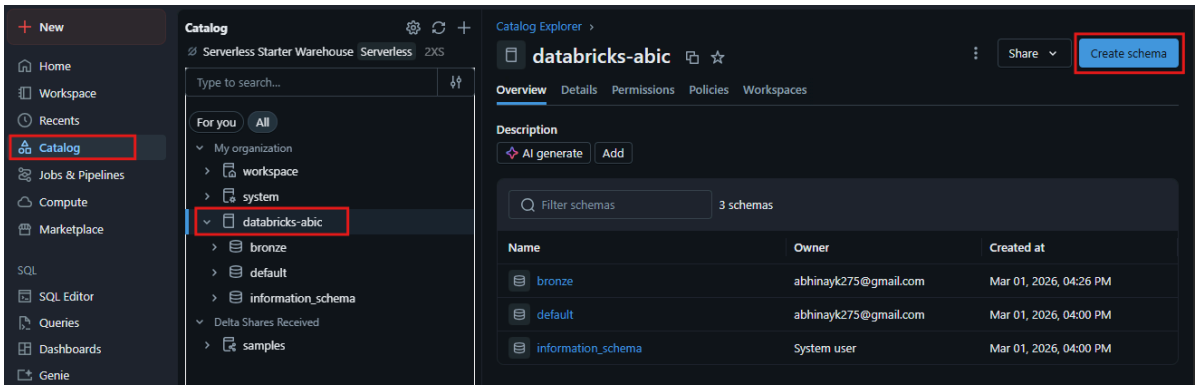
Step 1: Import DeltaTable

```
from delta.tables import DeltaTable
```

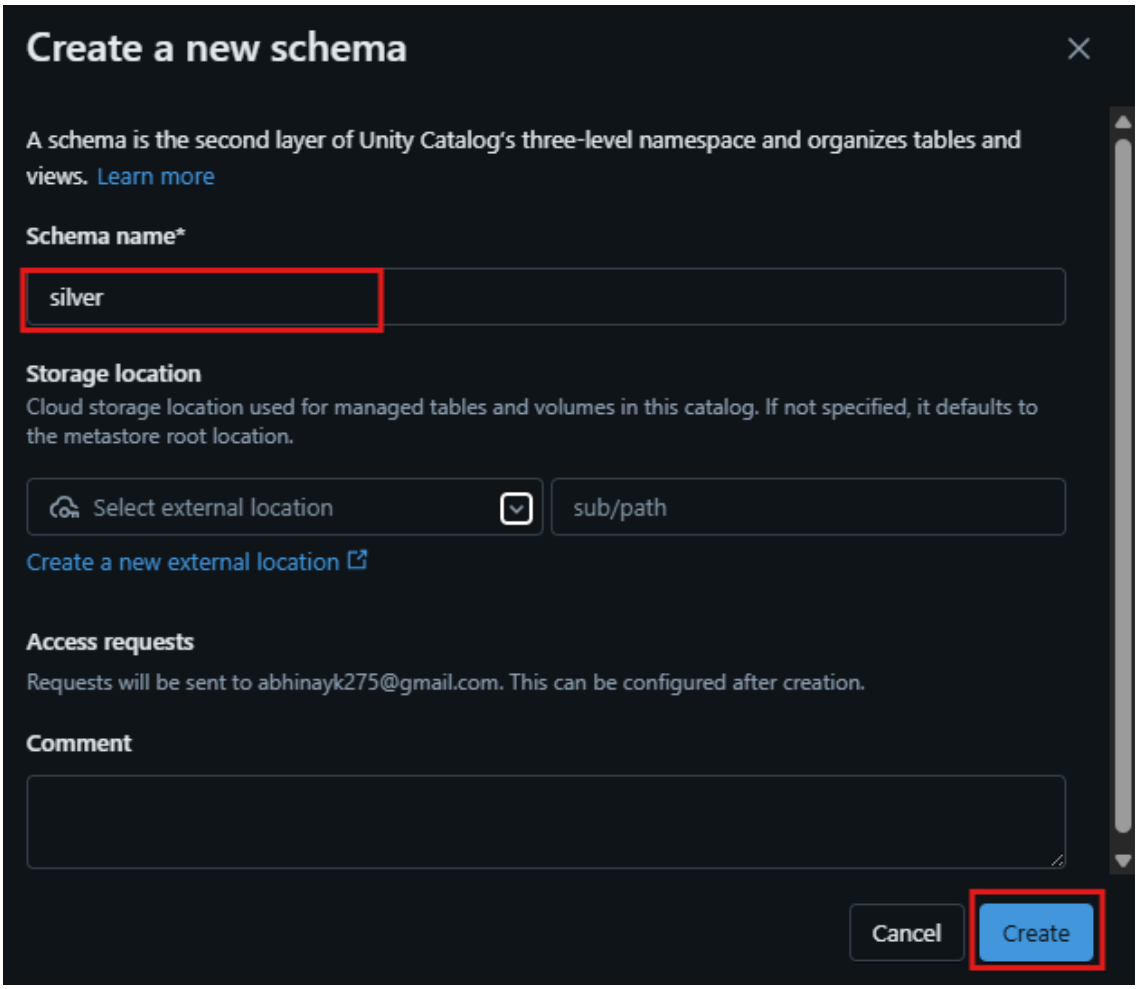


Step 2: Create Schema

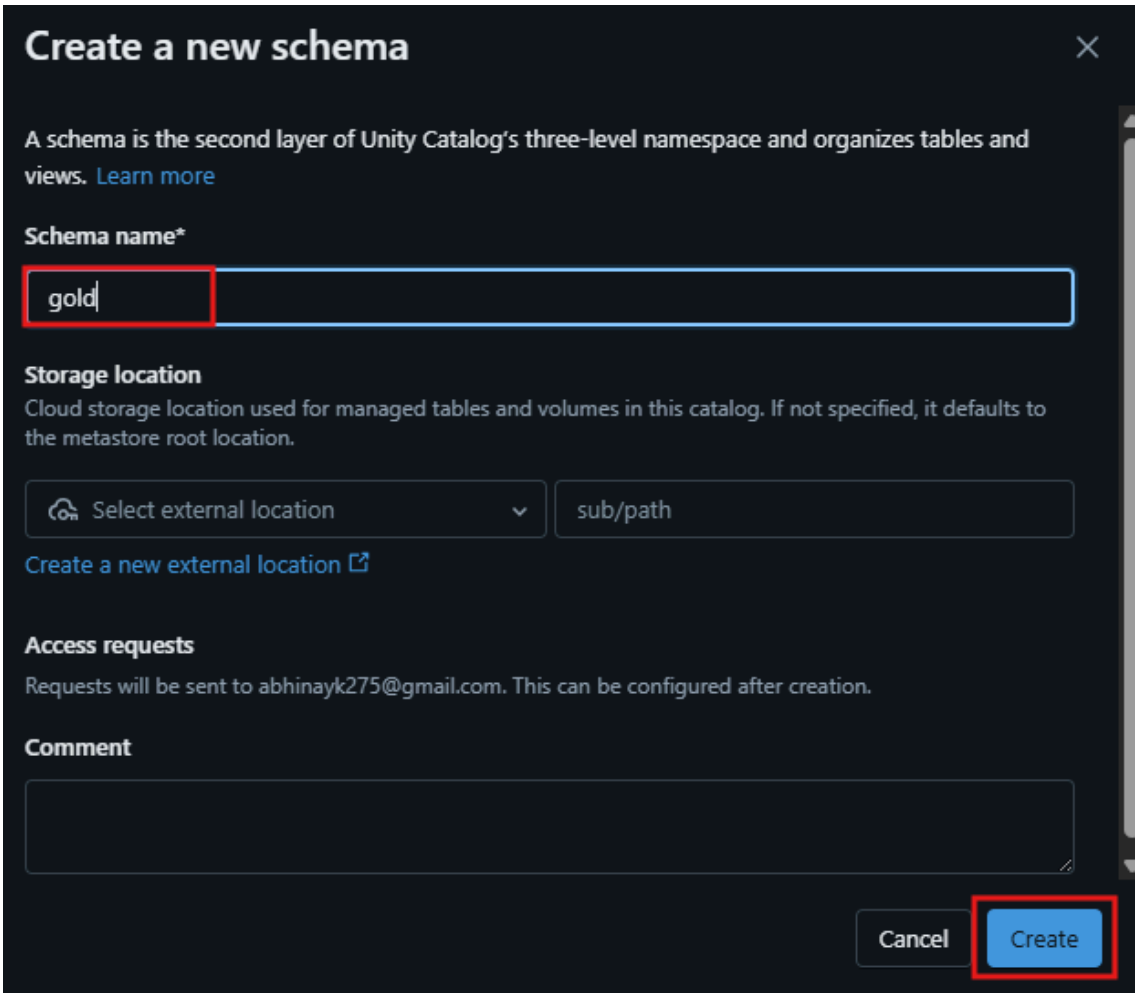
- Navigate to catalog —→ create new schemas



  - silver


Create a new schema
A schema is the second layer of Unity Catalog's three-level namespace and organizes tables and views. Learn more
Schema name*
silver
Storage location
Cloud storage location used for managed tables and volumes in this catalog. If not specified, it defaults to the metastore root location.
Select external location
sub/path
Create a new external location
Access requests
Requests will be sent to abhinayk275@gmail.com. This can be configured after creation.
Comment
Cancel
Create

- gold



Step A: Check If Table Exists

```
if spark.catalog.tableExists("databricks-abic.silver.customer
s_enriched"):
```

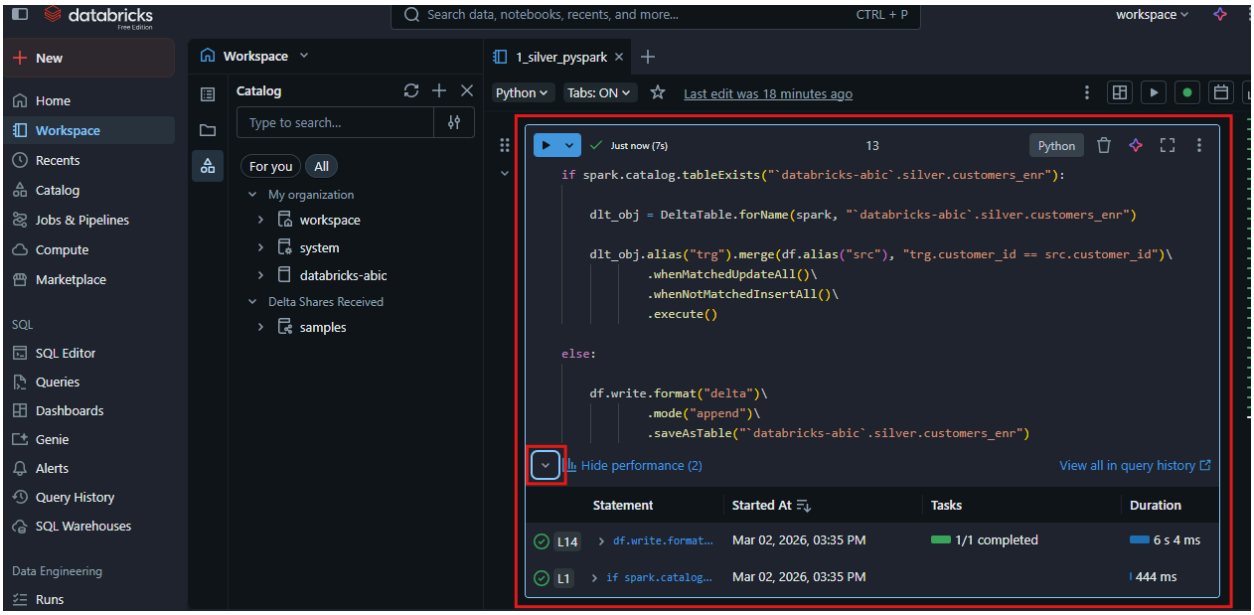
Step B: If Exists → Apply Merge

```
    delta_table = DeltaTable.forName(
        spark,
        "databricks-abic.silver.customers_enriched"
    )

    delta_table.alias("tgt").merge(
```

```
        df.alias("src"),
        "tgt.customer_id = src.customer_id"
    ).whenMatchedUpdateAll() \
     .whenNotMatchedInsertAll() \
     .execute()
```

Step C: If Not Exists → Create Table

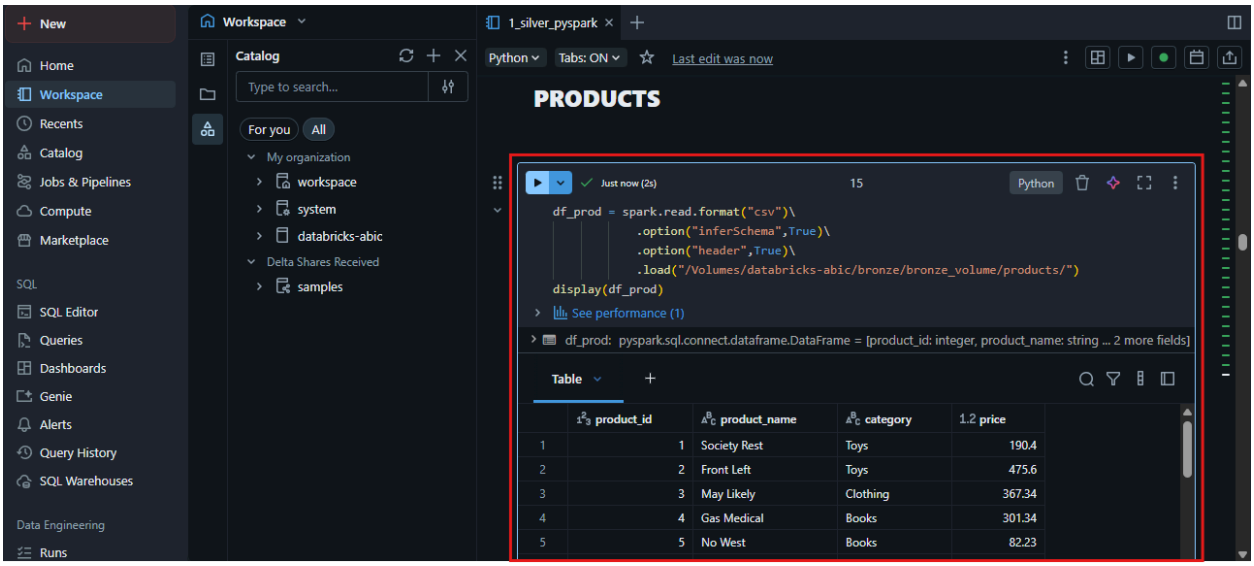
```
else:
    df.write \
      .format("delta") \
      .mode("append") \
      .saveAsTable("databricksansh.silver.customers_enriche
d")
```



Task 4: Process Products Data

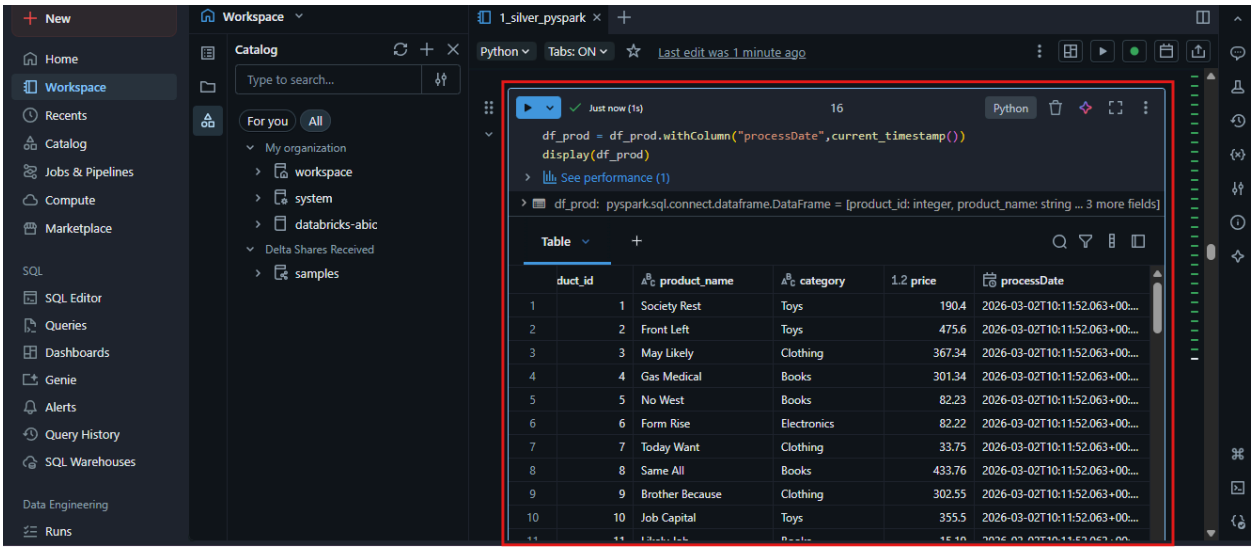
Step 4.1: Read Products

```
df_prod = spark.read \
    .format("csv") \
    .option("inferSchema", True) \
    .option("header", True) \
    .load("/Volumes/databricks-abic/bronze/bronze_volume/prod
ucts/")
```



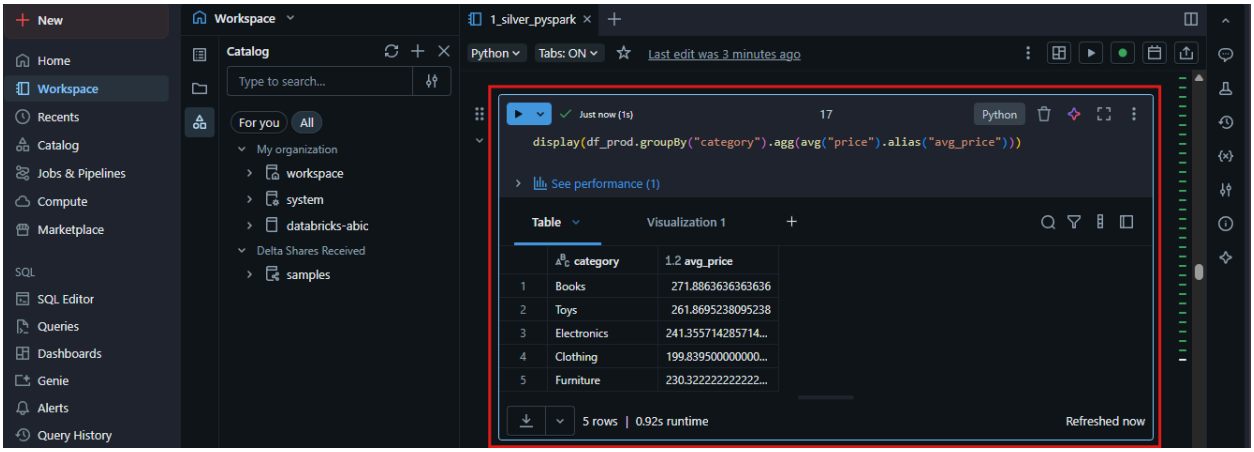
Step 4.2: Add Process Date

```
df_prod = df_prod.withColumn(
    "process_date",
    current_timestamp()
)
```

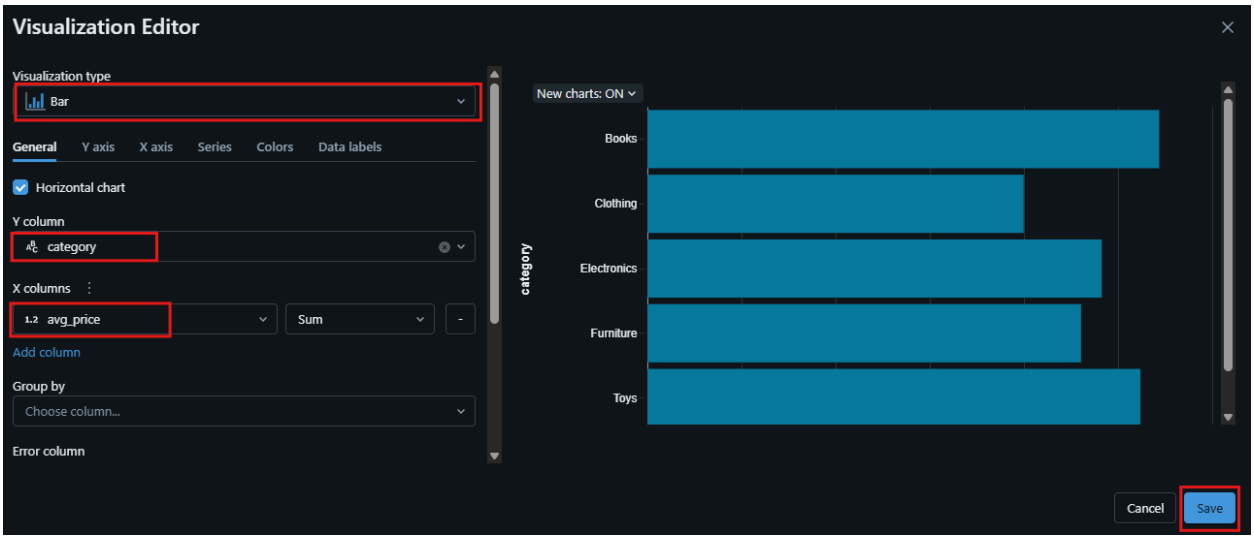


Step 4.3: Aggregation (Average Price per Category)

```
display(
    df_prod.groupBy("category")
           .agg(avg("price").alias("avg_price"))
)
```



Create Bar Chart Visualization.



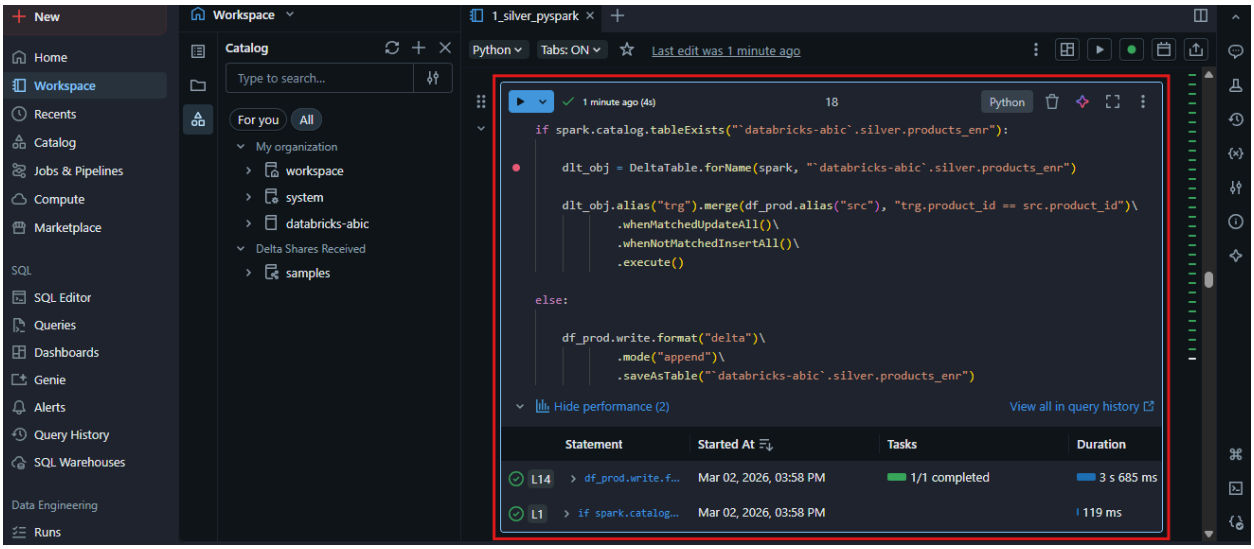
Step 4.4: Apply Upsert Logic

Primary Key → `product_id`

Same merge pattern as customers:

```
if spark.catalog.tableExists("databricks-abic.silver.products
_enriched"):

    delta_table = DeltaTable.forName(
        spark,
        "databricks-abic.silver.products_enriched"
    )

    delta_table.alias("tgt").merge(
        df_prod.alias("src"),
        "tgt.product_id = src.product_id"
    ).whenMatchedUpdateAll() \
     .whenNotMatchedInsertAll() \
     .execute()

else:
    df_prod.write \
        .format("delta") \
        .mode("append") \
```

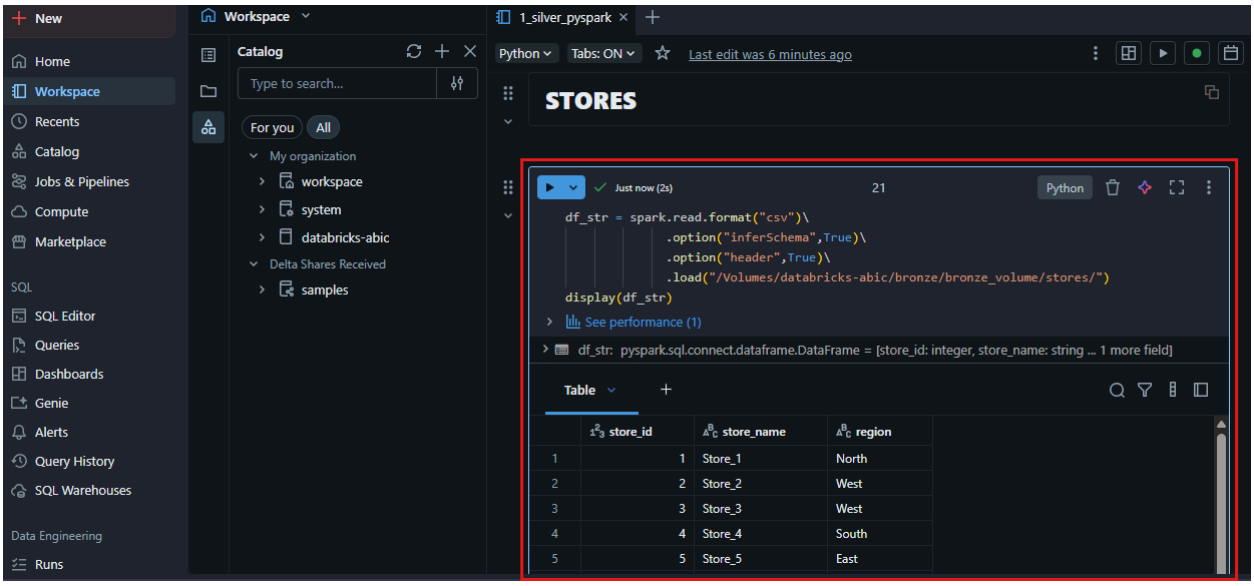
```
        .saveAsTable("databricks-abic.silver.products_enriche
d")
```



Task 5: Process Stores Data

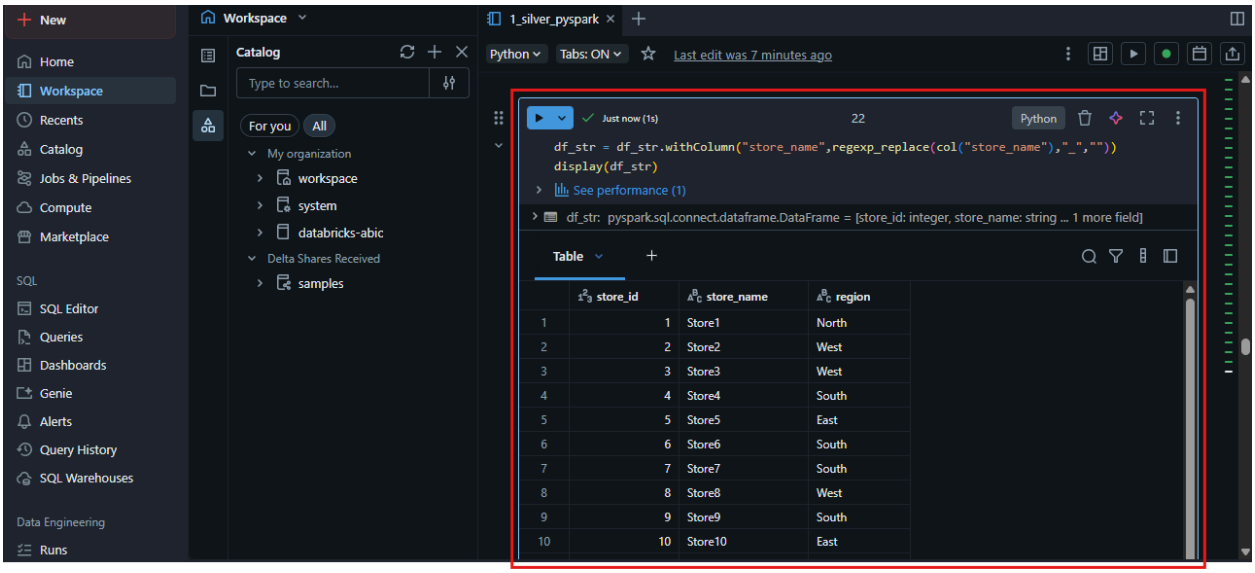
Step 5.1: Read Stores

```
df_str = spark.read \
    .format("csv") \
    .option("inferSchema", True) \
    .option("header", True) \
    .load("/Volumes/databricks-abic/bronze/bronze_volume/stor
es/")
```



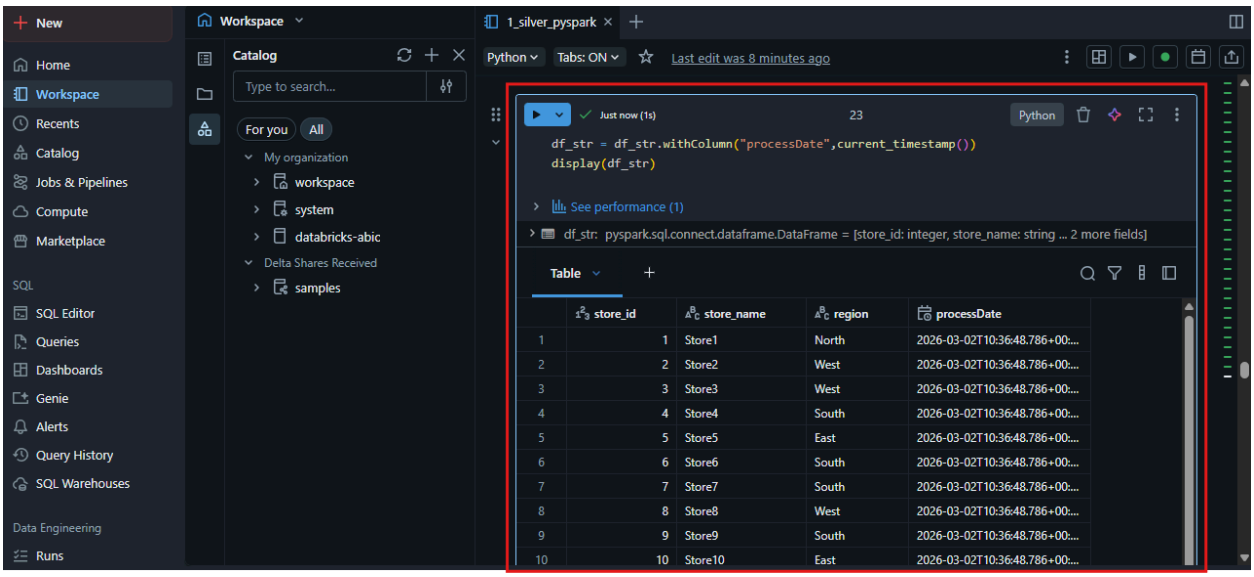
Step 5.2: Remove Underscore from Store Name

```
df_str = df_str.withColumn(
    "store_name",
    regexp_replace(col("store_name"), "_", "")
)
```



Step 5.3: Add Process Date

```
df_str = df_str.withColumn(
    "process_date",
    current_timestamp()
)
```



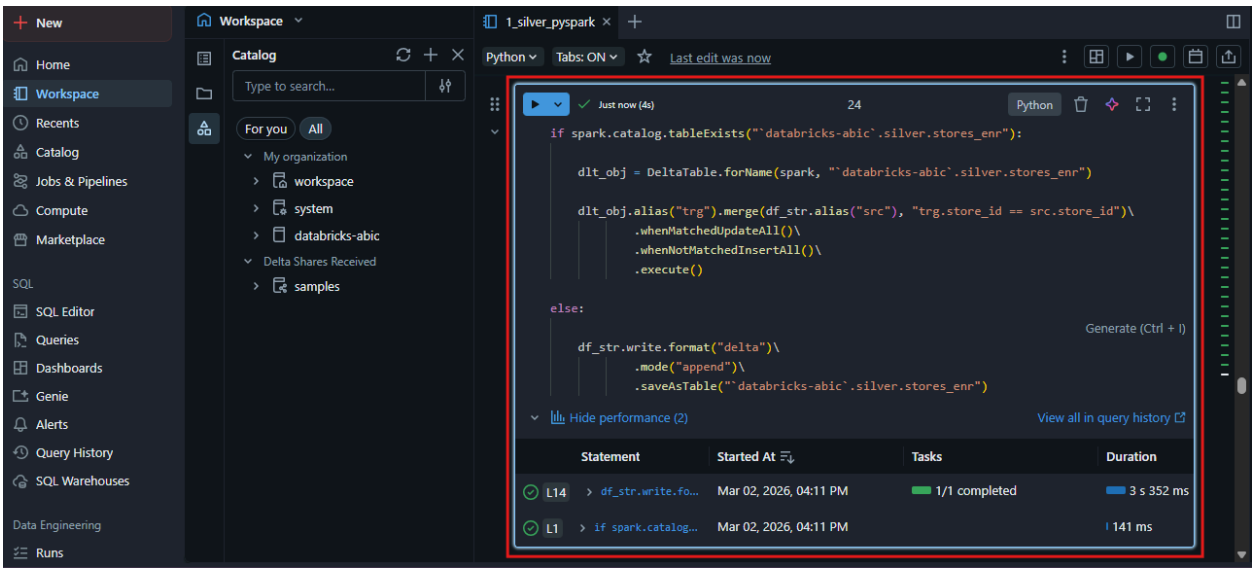
Step 5.4: Apply Upsert

Primary Key → `store_id`

Same merge logic as above.

```
if spark.catalog.tableExists("databricks-abic.silver.stores_enr"):

    dlt_obj = DeltaTable.forName(spark, "databricks-abic.silver.stores_enr")

    dlt_obj.alias("trg").merge(df_str.alias("src"), "trg.store_id == src.store_id")\
            .whenMatchedUpdateAll()\
            .whenNotMatchedInsertAll()\
            .execute()
```

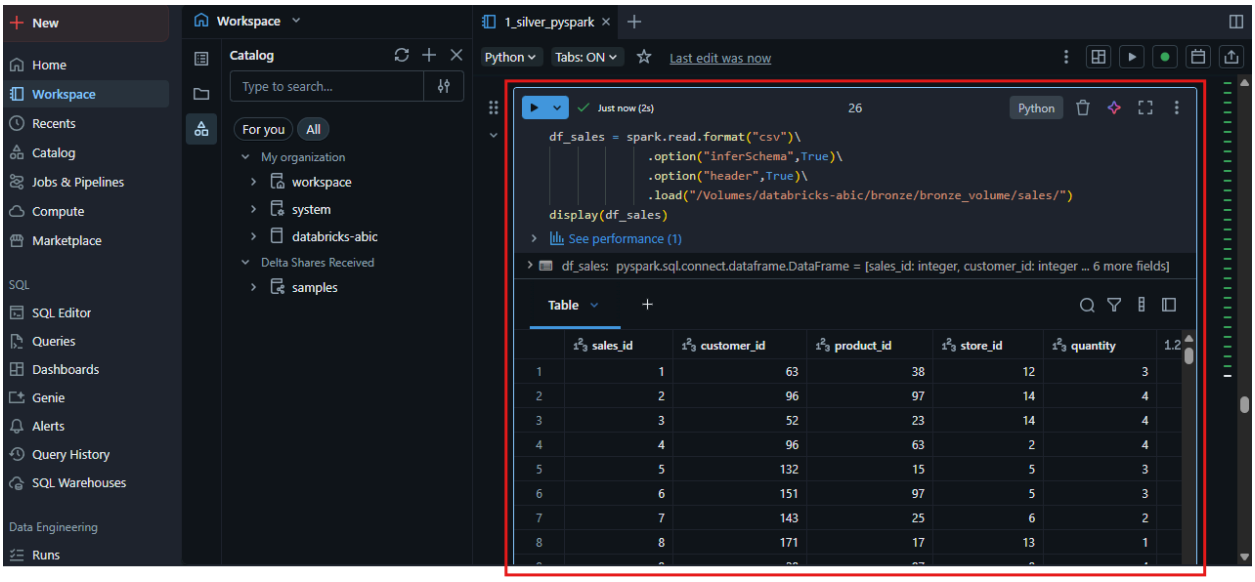
```
else:

    df_str.write.format("delta")\
            .mode("append")\
            .saveAsTable("databricks-abic.silver.stores_enr")
```



Task 6: Process Sales Data (Fact Table)

Step 6.1: Read Sales

```
df_sales = spark.read \
    .format("csv") \
    .option("inferSchema", True) \
    .option("header", True) \
    .load("/Volumes/databricks-abic/bronze/bronze_volume/sales/")
```

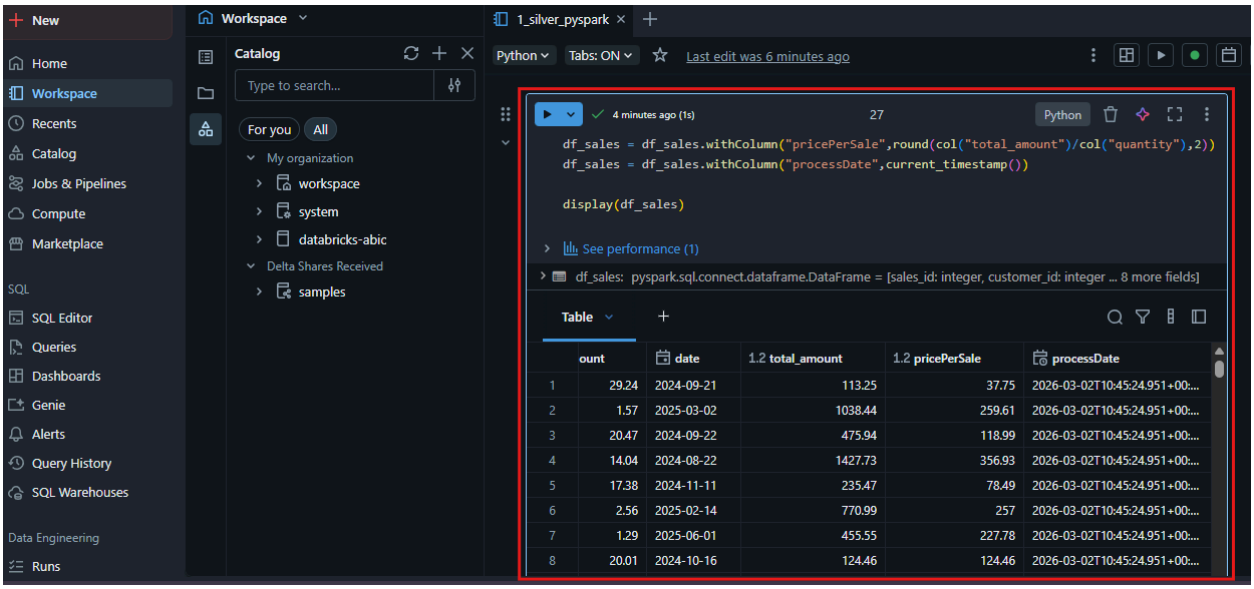


Step 6.2: Create Price Per Sale Column

```
df_sales = df_sales.withColumn(
    "price_per_sale",
    round(col("total_amount") / col("quantity"), 2)
)
```

Step 6.3: Add Process Date

```
df_sales = df_sales.withColumn(
    "process_date",
    current_timestamp()
)
```



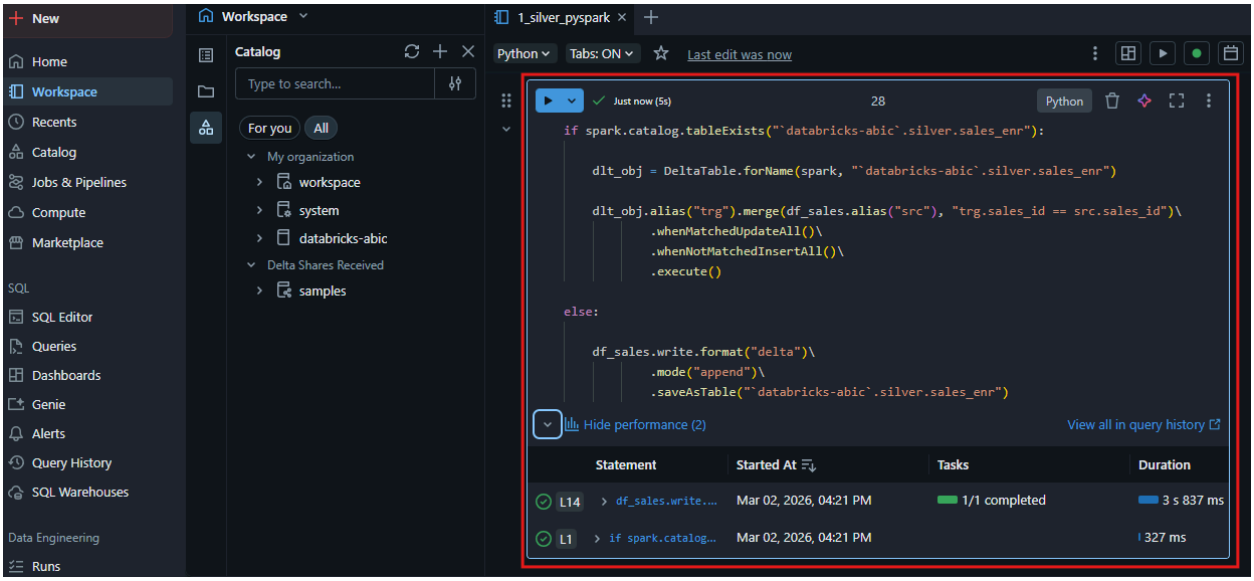
Step 6.4: Apply Upsert

Primary Key → `sale_id`

Use same merge pattern.

```
if spark.catalog.tableExists("`databricks-abic`.silver.sales_enr"):

    dlt_obj = DeltaTable.forName(spark, "`databricks-abic`.silver.sales_enr")

    dlt_obj.alias("trg").merge(df_sales.alias("src"), "trg.sales_id == src.sales_id")\
            .whenMatchedUpdateAll()\
            .whenNotMatchedInsertAll()\
            .execute()

else:

    df_sales.write.format("delta")\
            .mode("append")\
```

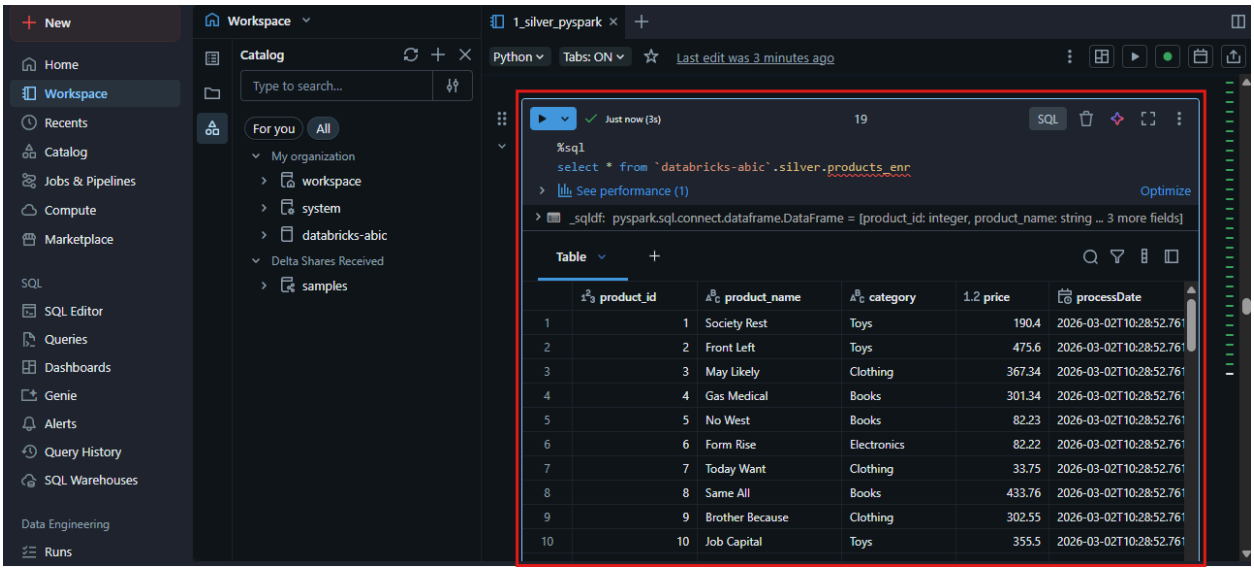
```
            .saveAsTable("`databricks-abic`.silver.sales_en
r")
```



Final Validation

Run:

```
SELECT COUNT(*) FROM databricks-abic.silver.customers_enr;
SELECT COUNT(*) FROM databricks-abic.silver.products_enr;
SELECT COUNT(*) FROM databricks-abic.silver.stores_enr;
SELECT COUNT(*) FROM databricks-abic.silver.sales_enr;
```



Re-run notebook and confirm record count does NOT increase (Upsert working correctly).

Outcome:

After completing this lab, you:

✔️ Understood Spark DataFrame Reader API

✔️ Performed column transformations

✔️ Used withColumn effectively

✔️ Applied string functions (upper, split, regexp_replace)

✔️ Performed aggregations

✔️ Created notebook visualizations

✔️ Added processing timestamp

✔️ Implemented Delta Merge (Upsert)

✔️ Built complete Silver layer tables

✔️ Prevented duplicate records